

AR Marker 3D Display



give detail steps for

AR Marker-Based 3D Model Display - Detect an image marker and show a 3D model (e.g., a car).

Here's a **complete step-by-step guide** to build an **AR Marker-Based 3D Model Display (Image Tracking → show 3D car model)** in Unity (works with Unity 2022+ and Unity 6.x using AR Foundation).

We will use:

- Unity
- AR Foundation
- ARCore XR Plugin (Android)
- ARKit XR Plugin (iOS)

PROJECT GOAL

When the camera detects a **printed image (marker)** → a **3D car model appears on it in real-world space**

STEP 1 — Create Unity Project

1. Open Unity Hub
2. Create New Project:
 - Template: **3D (URP optional)**
 - Name: `AR_Marker_Display`

STEP 2 — Install Required Packages

Go to:

Window → Package Manager

Install:

- AR Foundation
- ARCore XR Plugin

- ARKit XR Plugin
 - Input System (optional but recommended)
-

STEP 3 — Enable AR in Project Settings

Go to:

Edit → Project Settings → XR Plug-in Management

1. Install XR Plug-in Management (if asked)
 2. Enable:
 -  ARCore (Android)
 -  ARKit (iOS if needed)
-

STEP 4 — Switch Platform

Go to:

File → Build Settings

- Select **Android**
 - Click **Switch Platform**
-

STEP 5 — Add AR Session Setup

In Hierarchy:

Right Click → XR

Add:

1. **AR Session**
2. **AR Session Origin (XR Origin)**

Inside XR Origin:

- You will see **AR Camera**
-

STEP 6 — Add Image Tracking Manager

Select **XR Origin**

Add Component:

- **AR Tracked Image Manager**

Now configure:

- Create a reference image library (next step)

STEP 7 — Create Image Library (MOST IMPORTANT)

Go to:

Assets → Right Click → Create → XR → Reference Image Library

Name: **MarkerLibrary**

Add Image:

- Click + **Add Image**
- Select your marker image (printed image)
- Set:
 - Name: **CarMarker**
 - Physical size: e.g. **0.2 meters**

 This image is what AR will detect

STEP 8 — Import 3D Car Model

You need a model:

- Import **.fbx** or **.obj** car model
OR
- Download from Unity Asset Store

Drag it into:

- Assets folder

STEP 9 — Create AR Prefab (Car Object)

1. Drag car model into scene
2. Adjust:
 - Scale (very important → usually 0.1 to 0.5)
 - Rotation (face forward)
3. Make it prefab:
 - Drag into Assets → create prefab

STEP 10 — Connect Prefab to Image Tracking

Create script:

ImageTracker.cs

```
C#
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

public class ImageTracker : MonoBehaviour
{
    public ARTrackedImageManager trackedImageManager;
    public GameObject carPrefab;

    private Dictionary<string, GameObject> spawnedObjects = new Dictionary<string, GameObject>();

    void OnEnable()
    {
        trackedImageManager.trackedImagesChanged += OnImageChanged;
    }

    void OnDisable()
    {
        trackedImageManager.trackedImagesChanged -= OnImageChanged;
    }

    void OnImageChanged(ARTrackedImagesChangedEventArgs eventArgs)
    {
        foreach (var image in eventArgs.added)
        {
            SpawnOrUpdate(image);
        }

        foreach (var image in eventArgs.updated)
        {
            SpawnOrUpdate(image);
        }
    }

    void SpawnOrUpdate(ARTrackedImage image)
    {
        string name = image.referenceImage.name;

        if (!spawnedObjects.ContainsKey(name))
        {
            GameObject obj = Instantiate(carPrefab, image.transform);
            spawnedObjects[name] = obj;
        }

        spawnedObjects[name].SetActive(image.trackingState == TrackingState.Tracking);

        spawnedObjects[name].transform.position = image.transform.position;
        spawnedObjects[name].transform.rotation = image.transform.rotation;
    }
}
```

STEP 11 — Assign References

In Unity:

1. Create empty GameObject → ARManager
2. Attach script ImageTracker

Then drag:

- AR Tracked Image Manager → into script field
 - Car Prefab → into script field
-

STEP 12 — Assign Reference Image Library

Select:

XR Origin → AR Tracked Image Manager

Set:

- **Reference Image Library = MarkerLibrary**
-

STEP 13 — Build Settings (IMPORTANT)

Go to:

Player Settings → Android

Set:

- Minimum API Level: 24+
- Scripting Backend: IL2CPP
- Target Architecture: ARM64

Enable:

- Camera permission
-

STEP 14 — Build & Run

1. Connect Android phone
 2. Enable USB debugging
 3. Click:
 - Build & Run
-

FINAL RESULT

When you:

- Open app
- Show printed marker image to camera

👉 A 3D car model appears perfectly on top of it

TROUBLESHOOTING

❌ Model not showing?

- Check scale (very small issue most of time)
- Ensure image is same as library

❌ Tracking not working?

- Good lighting needed
- Marker must be clear (not blurred)

❌ Camera black screen?

- Check permission + ARCore enabled

BONUS IDEAS

You can upgrade this project with:

- Car rotation animation
- Tap to change car color
- Multiple markers → multiple models
- Sound effects when detected

If you want, I can also give:

- ✓ Full Unity package structure
- ✓ Ready-made scripts + prefab setup
- ✓ OR AR VR combined project (marker + VR mode switching)

Just tell 👍



AR Flashcard Learning App - Scan alphabet flashcards to display corresponding 3D models (A-E)

Here's a **complete step-by-step guide** to build an **AR Flashcard Learning App (A-E alphabets → 3D models appear on scan)** using Unity + AR Foundation.

You'll create an app where:

- You scan **alphabet flashcards (A, B, C, D, E)**
- Each card shows a **3D object (Apple, Ball, Cat, Dog, Elephant, etc.)**

TECH STACK

- Unity
- AR Foundation
- ARCore XR Plugin
- ARKit XR Plugin

PROJECT IDEA

Flashcard	Model
A	Apple 🍏
B	Ball ⚽
C	Cat 🐱
D	Dog 🐶
E	Elephant 🐘

STEP 1 — Create Unity Project

1. Open Unity Hub
2. Create project:
 - Template: **3D (URP optional)**
 - Name: **AR_Flashcard_App**

STEP 2 — Install Packages

Go to:

Window → Package Manager

Install:

- AR Foundation
- ARCore XR Plugin
- ARKit XR Plugin

Also install:

- Input System (recommended)

STEP 3 — Enable AR

Go to:

Edit → Project Settings → XR Plug-in Management

Enable:

-  ARCore (Android)
-  ARKit (iOS optional)

STEP 4 — Switch Platform

Go to:

File → Build Settings

- Select Android → Switch Platform

STEP 5 — Add AR Scene Setup

Hierarchy:

Right Click → XR

Add:

- AR Session
- AR Session Origin (XR Origin)

Inside XR Origin:

- AR Camera (already present)

STEP 6 — Add Image Tracking

Select **XR Origin**

Add component:

- **AR Tracked Image Manager**

STEP 7 — Create Flashcard Library

Go to:

Assets → Create → XR → Reference Image Library

Name: **FlashcardLibrary**

Add Images (IMPORTANT)

Add 5 images:

Name	Image
A	A flashcard image
B	B flashcard image
C	C flashcard image
D	D flashcard image
E	E flashcard image

👉 Make sure each image is:

- Clear
- High contrast
- Printed or shown on screen

STEP 8 — Import 3D Models

You need 5 models:

- Apple
- Ball
- Cat
- Dog
- Elephant

You can get from:

- Unity Asset Store
- Sketchfab (free models)

Drag them into Assets.

STEP 9 — Create Prefabs

For each model:

1. Drag into Scene
2. Adjust scale (VERY IMPORTANT)
3. Drag back into Assets → create prefab

You will have:

- ApplePrefab
- BallPrefab
- CatPrefab
- DogPrefab
- ElephantPrefab

STEP 10 — Create Script (Core Logic)

Create script:

FlashcardARManager.cs

```
C#
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

public class FlashcardARManager : MonoBehaviour
{
    public ARTrackedImageManager imageManager;

    public GameObject applePrefab;
    public GameObject ballPrefab;
    public GameObject catPrefab;
    public GameObject dogPrefab;
    public GameObject elephantPrefab;

    private Dictionary<string, GameObject> spawnedObjects = new Dictionary<string, GameObject>();

    void OnEnable()
    {
        imageManager.trackedImagesChanged += OnImagesChanged;
    }

    void OnDisable()
    {
        imageManager.trackedImagesChanged -= OnImagesChanged;
    }

    void OnImagesChanged(ARTrackedImagesChangedEventArgs eventArgs)
    {
        foreach (var img in eventArgs.added)
        {
            UpdateImage(img);
        }

        foreach (var img in eventArgs.updated)
        {
            UpdateImage(img);
        }
    }
}
```

```

void UpdateImage(ARTrackedImage img)
{
    string name = img.referenceImage.name;

    GameObject prefab = GetPrefab(name);

    if (prefab == null) return;

    if (!spawnedObjects.ContainsKey(name))
    {
        GameObject obj = Instantiate(prefab, img.transform);
        spawnedObjects[name] = obj;
    }

    spawnedObjects[name].SetActive(img.trackingState == TrackingState.Tracking);

    spawnedObjects[name].transform.position = img.transform.position;
    spawnedObjects[name].transform.rotation = img.transform.rotation;
}

GameObject GetPrefab(string name)
{
    switch (name)
    {
        case "A": return applePrefab;
        case "B": return ballPrefab;
        case "C": return catPrefab;
        case "D": return dogPrefab;
        case "E": return elephantPrefab;
    }
    return null;
}
}

```

STEP 11 — Setup in Unity

1. Create empty GameObject:

- Name: ARManager

2. Attach script:

- FlashcardARManager

3. Drag references:

- AR Tracked Image Manager
- All 5 prefabs

STEP 12 — Assign Image Library

Select:

- XR Origin → AR Tracked Image Manager

Set:

- Reference Image Library → FlashcardLibrary

STEP 13 — Player Settings

Go to:

Player Settings (Android)

Set:

- Minimum API Level: 24+
- Scripting Backend: IL2CPP
- Architecture: ARM64

Enable:

- Camera permission

STEP 14 — Build & Run

1. Connect phone
2. Enable USB debugging
3. Click:
 - Build & Run

FINAL RESULT

When you:

- Open app
- Show flashcard "A"

👉 Apple 🍏 appears in AR
 Show "B" → Ball ⚽ appears
 Show "C" → Cat 🐱 appears
 ...and so on

UPGRADES (VERY GOOD FOR PROJECT MARKS)

You can enhance it with:

- 🔊 Voice: "A for Apple"
- ✨ Particle effects on detection
- 📖 Quiz mode ("What is this?")
- 🖱️ Tap object → animation
- 🎨 UI showing alphabet name

⚠ COMMON ISSUES

✗ Nothing appears?

- Check image library names match script exactly (A, B, C...)

✗ Tracking unstable?

- Improve lighting
- Use high-quality printed flashcards

✗ Model too big/small?

- Adjust prefab scale (usually 0.05–0.3)

If you want next level upgrade, I can help you build:

- ✓ Voice-enabled AR learning app
- ✓ Offline AR flashcard game with scoring
- ✓ Or full Unity project ZIP structure ready to submit

Just tell 👍



AR Business Card - Scan a business card to show LinkedIn, phone, email in 3D overlay

Here's a **complete, project-ready step-by-step guide** to build an **AR Business Card App** where scanning a physical business card shows a **3D overlay with LinkedIn, phone, email, and profile info**.

You'll build this using:

- Unity
- AR Foundation
- ARCore XR Plugin (Android)
- ARKit XR Plugin (iOS)

🎯 PROJECT IDEA

When user scans a **printed business card image**, a **3D info panel appears above it**:

Example overlay:

- 👤 Name
- 💼 Role

-  Phone
-  Email
-  LinkedIn button (clickable)

STEP 1 — Create Unity Project

1. Open Unity Hub
2. Create project:
 - Template: **3D (URP optional)**
 - Name: **AR_BusinessCard**

STEP 2 — Install Packages

Open:

Window → Package Manager

Install:

- AR Foundation
- ARCore XR Plugin
- ARKit XR Plugin

Also install:

- Input System (optional)

STEP 3 — Enable AR

Go to:

Edit → Project Settings → XR Plug-in Management

Enable:

-  ARCore (Android)
-  ARKit (iOS optional)

STEP 4 — Switch Platform

File → Build Settings

- Select Android → Switch Platform

STEP 5 — Add AR Setup

Hierarchy:

XR → Add:

- AR Session
- AR Session Origin (XR Origin)

Inside XR Origin:

- AR Camera (auto created)

STEP 6 — Enable Image Tracking

Select **XR Origin**

Add component:

- AR Tracked Image Manager

STEP 7 — Create Business Card Image Library

Go to:

Assets → Create → XR → Reference Image Library

Name: **BusinessCardLibrary**

Add your business card image

- Add image of your printed business card
- Name it: **Card**

 This image is what AR will detect

STEP 8 — Create 3D UI Card (Overlay)

Now we create floating info panel.

Create UI:

1. Right click → 3D Object → **Quad**
2. Name: **InfoPanel**

3. Make it a prefab

Add Text Elements:

On Quad, add:

- Name Text
- Role Text
- Phone Text
- Email Text
- LinkedIn Button (optional cube or UI button)

Use World Space UI if needed.

Example Layout (3D Panel)

```
[ Name: Vishnu ]
[ Role: Developer ]
[ Phone: +91 XXXXX ]
[ Email: abc@gmail.com ]
[ LinkedIn Button ]
```

STEP 9 — Create Script

BusinessCardAR.cs

```
C#
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

public class BusinessCardAR : MonoBehaviour
{
    public ARTrackedImageManager imageManager;
    public GameObject infoPanelPrefab;

    private Dictionary<string, GameObject> spawnedPanels = new Dictionary<string, GameObject>();

    void OnEnable()
    {
        imageManager.trackedImagesChanged += OnImageChanged;
    }

    void OnDisable()
    {
        imageManager.trackedImagesChanged -= OnImageChanged;
    }

    void OnImageChanged(ARTrackedImagesChangedEventArgs eventArgs)
    {
        foreach (var img in eventArgs.added)
            UpdateCard(img);

        foreach (var img in eventArgs.updated)
            UpdateCard(img);
    }
}
```



```
void UpdateCard(ARTrackedImage img)
{
    string name = img.referenceImage.name;

    if (!spawnedPanels.ContainsKey(name))
    {
        GameObject panel = Instantiate(infoPanelPrefab, img.transform);
        spawnedPanels[name] = panel;

        // Set business info
        panel.transform.Find("Name").GetComponent<TextMesh>().text = "John Doe";
        panel.transform.Find("Role").GetComponent<TextMesh>().text = "Software Engineer";
        panel.transform.Find("Phone").GetComponent<TextMesh>().text = "+91 9876543210";
        panel.transform.Find("Email").GetComponent<TextMesh>().text = "john@email.com";
    }

    spawnedPanels[name].SetActive(img.trackingState == TrackingState.Tracking);

    spawnedPanels[name].transform.position = img.transform.position + new Vector3(0, 0.1f, 0);
    spawnedPanels[name].transform.rotation = img.transform.rotation;
}
}
```

STEP 10 — Add LinkedIn Button (Optional)

To open LinkedIn:

```
C#
public void OpenLinkedIn()
{
    Application.OpenURL("https://linkedin.com/in/yourprofile");
}
```

Attach to button.

STEP 11 — Setup in Unity

1. Create empty GameObject → **ARManager**
2. Attach script **BusinessCardAR**

Assign:

- AR Tracked Image Manager
- Info Panel Prefab

STEP 12 — Assign Image Library

Select:

- XR Origin → AR Tracked Image Manager

Set:

- BusinessCardLibrary

STEP 13 — Player Settings

Go to Android settings:

- Min API: 24+
- Scripting Backend: IL2CPP
- Architecture: ARM64
- Enable Camera permission

STEP 14 — Build & Run

1. Connect phone
2. Enable USB debugging
3. Build & Run

FINAL RESULT

When you scan a business card:

 A floating 3D panel appears showing:

- Name
- Role
- Phone
- Email
- LinkedIn link

It stays anchored on the card in real-world space.

IMPROVEMENTS (FOR IMPRESSIVE PROJECT)

You can upgrade it with:

- ✨ Animated card entry (fade-in)
- 🔊 Voice greeting (“Hello, I am John”)
- 📞 Tap to call phone number
- 📧 Email button opens Gmail app
- 🎯 Multiple business cards support
- 🌐 QR code + AR hybrid system

⚠ COMMON ISSUES

✗ Panel not appearing?

- Check image name matches library exactly

✗ Tracking unstable?

- Improve lighting
- Avoid shiny/low contrast cards

✗ UI not readable?

- Increase panel scale
- Use simple fonts

If you want next upgrade, I can help you build:

- ✓ Fully animated AR visiting card (like Apple style)
- ✓ Multi-user digital networking AR app
- ✓ Or a complete portfolio project ready for placement

Just tell 👍



AR Greeting Card with 3D Animation: Design an AR greeting card where scanning the card with a mobile device shows a 3D animation or message. For example, when a birthday card is scanned, balloons and confetti appear in the AR scene.

Here's a **complete, step-by-step guide** to build an **AR Greeting Card with 3D Animation (Birthday Card → balloons + confetti + message)** using Unity.

You'll create an app where:

- User scans a **printed greeting card image**
- A **3D animated celebration scene appears** (balloons, confetti, message)

🎯 PROJECT OVERVIEW

When a birthday card is scanned:

- 🎈 Balloons float up
- 🎊 Confetti bursts
- 🎂 "Happy Birthday" message appears

- 🎵 Optional background music plays

Built using:

- Unity
 - AR Foundation
 - ARCore XR Plugin
 - ARKit XR Plugin
-

STEP 1 — Create Unity Project

1. Open Unity Hub
 2. Create new project:
 - Template: **3D**
 - Name: **AR_GreetingCard**
-

STEP 2 — Install Required Packages

Go to:

Window → Package Manager

Install:

- AR Foundation
- ARCore XR Plugin
- ARKit XR Plugin

(Optional)

- Input System
-

STEP 3 — Enable AR

Go to:

Edit → Project Settings → XR Plug-in Management

Enable:

- ✓ ARCore (Android)
 - ✓ ARKit (iOS optional)
-

STEP 4 — Switch Platform

Go to:

File → Build Settings

- Switch to Android
-

STEP 5 — Add AR Setup

In Hierarchy:

XR → Add:

- AR Session
- AR Session Origin (XR Origin)

Inside XR Origin:

- AR Camera
-

STEP 6 — Enable Image Tracking

Select XR Origin:

Add component:

- AR Tracked Image Manager
-

STEP 7 — Create Greeting Card Image Library

Go to:

Assets → Create → XR → Reference Image Library

Name: `GreetingCardLibrary`

Add Image:

- Add your **birthday card image**
- Name: `BirthdayCard`

 This is the trigger image

STEP 8 — Create AR Celebration Scene

We will create:

 **Balloons**

 **Confetti**

 **Message Text**

Balloons

1. Create → 3D Object → Sphere
 2. Scale it (small balloon shape)
 3. Add color material (red, blue, yellow)
 4. Add **Rigidbody**
 - Use Gravity = OFF
 5. Add script for floating motion
-

Confetti

Option 1 (best):

- Particle System

Create:

- GameObject → Effects → Particle System

Settings:

- Start speed: high
 - Gravity: low
 - Color: random
 - Shape: Cone or Box
-

Message

1. Create → 3D Text (TextMeshPro)
2. Text:

"Happy Birthday!"
3. Style:
 - Big font

- Bright color (yellow/pink)

STEP 9 — Make Prefab

Select all:

- Balloons
- Confetti
- Text

Drag into Assets → create prefab:

 BirthdayARPrefab

STEP 10 — AR Script

GreetingCardAR.cs

```
C#
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

public class GreetingCardAR : MonoBehaviour
{
    public ARTrackedImageManager imageManager;
    public GameObject birthdayPrefab;

    private Dictionary<string, GameObject> spawnedObjects = new Dictionary<string, GameObject>();

    void OnEnable()
    {
        imageManager.trackedImagesChanged += OnImageChanged;
    }

    void OnDisable()
    {
        imageManager.trackedImagesChanged -= OnImageChanged;
    }

    void OnImageChanged(ARTrackedImagesChangedEventArgs eventArgs)
    {
        foreach (var img in eventArgs.added)
            UpdateAR(img);

        foreach (var img in eventArgs.updated)
            UpdateAR(img);
    }

    void UpdateAR(ARTrackedImage img)
    {
        string name = img.referenceImage.name;

        if (!spawnedObjects.ContainsKey(name))
        {
            GameObject obj = Instantiate(birthdayPrefab, img.transform);
            spawnedObjects[name] = obj;
        }

        spawnedObjects[name].SetActive(img.trackingState == TrackingState.Tracking);
        spawnedObjects[name].transform.position = img.transform.position;
    }
}
```

```

        spawnedObjects[name].transform.rotation = img.transform.rotation;
    }
}

```

STEP 11 — Setup in Unity

1. Create empty GameObject → **ARManager**
2. Attach script **GreetingCardAR**

Assign:

- AR Tracked Image Manager
- BirthdayARPrefab

STEP 12 — Assign Image Library

Select:

- XR Origin → AR Tracked Image Manager

Set:

- GreetingCardLibrary

STEP 13 — Add Animation (IMPORTANT)

Balloon Floating Script

Attach to balloons:

```

C#
using UnityEngine;

public class BalloonFloat : MonoBehaviour
{
    void Update()
    {
        transform.position += new Vector3(0, Time.deltaTime * 0.2f, 0);
        transform.Rotate(0, 20 * Time.deltaTime, 0);
    }
}

```

STEP 14 — Add Confetti Burst

On spawn:

- Particle System → Play automatically
- Duration: 3–5 sec burst

STEP 15 — Player Settings

Go to Android settings:

- Min API Level: 24+
 - Scripting Backend: IL2CPP
 - ARM64 enabled
 - Camera permission ON
-

STEP 16 — Build & Run

1. Connect phone
 2. Enable USB debugging
 3. Build & Run
-

FINAL RESULT

When user scans birthday card:

- 👉 🎈 Balloons rise up
- 👉 🎊 Confetti explodes
- 👉 🎂 "Happy Birthday" appears
- 👉 🎵 Optional music plays

All anchored on the physical card in real time.

IMPROVEMENTS (FOR HIGH MARKS)

You can upgrade it to:

- 🎵 Background birthday song
 - 📺 Gift box animation opening
 - ✨ Sparkle effects on text
 - 📷 Screenshot button
 - 🎂 Multiple greeting card types (Birthday, Anniversary, etc.)
 - 👤 Personalized name input ("Happy Birthday Rahul")
-

COMMON ISSUES

Nothing appears

- Image name mismatch

- Wrong reference library assigned

✗ Tracking unstable

- Use high-quality printed card
- Good lighting required

✗ Animation not working

- Check prefab parenting
- Ensure scripts attached properly

If you want next upgrade:

I can help you build:

- ✓ Fully animated AR greeting card with UI menu
- ✓ Personalized name input system (dynamic messages)
- ✓ Or a full AR gift app (multi-card system + QR unlock)

Just tell 👍



AR Image Recognition and Object Placement Create an AR app that recognizes an image (such as a logo or a specific picture) and places a 3D object (like an animal or character) on top of it when the image is detected.

Here's a **complete, structured step-by-step guide** to build an **AR Image Recognition + 3D Object Placement App** (logo/picture → spawn animal/character on top).

You'll use:

- Unity
- AR Foundation
- ARCore XR Plugin (Android)
- ARKit XR Plugin (iOS)

PROJECT GOAL

When camera detects a **specific image (logo/photo)**:

- 👉 A **3D object (animal/character)** appears on top of it
- 👉 Object stays fixed to the image in real world

Example:

- Nike logo → Tiger appears 🐯
 - Company logo → 3D mascot appears
 - Printed photo → Animated character appears
-

STEP 1 — Create Unity Project

1. Open Unity Hub
 2. New Project:
 - Template: **3D (Core)**
 - Name: `AR_ImageObjectPlacement`
-

STEP 2 — Install Packages

Open:

Window → Package Manager

Install:

- AR Foundation
- ARCore XR Plugin
- ARKit XR Plugin

(Optional)

- Input System
-

STEP 3 — Enable AR

Go to:

Edit → Project Settings → XR Plug-in Management

Enable:

- ☒ ARCore (Android)
 - ☒ ARKit (iOS optional)
-

STEP 4 — Switch Platform

Go to:

File → Build Settings

- Switch to Android

STEP 5 — Setup AR Scene

In Hierarchy:

XR → Add:

- AR Session
- AR Session Origin (XR Origin)

Inside XR Origin:

- AR Camera

STEP 6 — Enable Image Tracking

Select XR Origin:

Add component:

- AR Tracked Image Manager

STEP 7 — Create Image Library

Go to:

Assets → Create → XR → Reference Image Library

Name: **ImageLibrary**

Add Image:

Add your:

- Logo OR picture OR printed image

Example:

Image Name	Meaning
Logo1	triggers object spawn

 IMPORTANT:

- High contrast image works best

- Flat printed image or screen image

STEP 8 — Import 3D Model

Import:

- Animal model (lion, tiger, dog)
- OR character model (Unity asset store / Sketchfab)

Then:

- Drag into scene
- Adjust scale (VERY IMPORTANT)
- Create prefab:
 **AnimalPrefab**

STEP 9 — AR Placement Script

ImageObjectSpawner.cs

```
C#
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

public class ImageObjectSpawner : MonoBehaviour
{
    public ARTrackedImageManager trackedImageManager;
    public GameObject objectPrefab;

    private Dictionary<string, GameObject> spawnedObjects = new Dictionary<string, GameObject>();

    void OnEnable()
    {
        trackedImageManager.trackedImagesChanged += OnImageChanged;
    }

    void OnDisable()
    {
        trackedImageManager.trackedImagesChanged -= OnImageChanged;
    }

    void OnImageChanged(ARTrackedImagesChangedEventArgs eventArgs)
    {
        foreach (var img in eventArgs.added)
            UpdateObject(img);

        foreach (var img in eventArgs.updated)
            UpdateObject(img);
    }

    void UpdateObject(ARTrackedImage img)
    {
        string name = img.referenceImage.name;

        if (!spawnedObjects.ContainsKey(name))
        {
            GameObject obj = Instantiate(objectPrefab, img.transform);
            spawnedObjects[name] = obj;
        }
    }
}
```

```

    }

    spawnedObjects[name].SetActive(img.trackingState == TrackingState.Tracking);

    spawnedObjects[name].transform.position = img.transform.position + new Vector3(0, 0.05f, 0);
    spawnedObjects[name].transform.rotation = img.transform.rotation;
}
}

```

STEP 10 — Setup in Unity

1. Create empty GameObject:

- Name: **ARManager**

2. Attach script:

- **ImageObjectSpawner**

3. Assign:

- AR Tracked Image Manager
- AnimalPrefab

STEP 11 — Assign Image Library

Select:

- XR Origin → AR Tracked Image Manager

Set:

- ImageLibrary

STEP 12 — Player Settings

Go to Android settings:

- Minimum API: 24+
- Scripting Backend: IL2CPP
- ARM64 enabled
- Camera permission ON

STEP 13 — Build & Run

1. Connect Android phone
2. Enable USB debugging
3. Click Build & Run

FINAL RESULT

When you scan image:

- 👉 Image is detected
- 👉 3D animal/character appears on top
- 👉 Object stays fixed while camera moves

Example:

- Logo → Lion appears roaring 🦁
- Poster → Character stands on it 🎮

IMPROVEMENTS (HIGH MARKS PROJECT)

You can upgrade it with:

✨ Animation

- Idle animation (walking, breathing)

Sound

- Animal sound when detected

Interaction

- Tap object → change animation

Multiple Objects

- Different images → different models

Screenshot feature

- Save AR scene image

COMMON PROBLEMS

✗ Object not appearing

- Check image name match exactly

✗ Tracking unstable

- Improve lighting
- Use printed high-quality images

✗ Object floating wrong position

- Adjust offset:

C#

```
+ new Vector3(0, 0.05f, 0)
```

NEXT LEVEL (IF YOU WANT)

I can help you build:

- ✓ Multi-image AR system (10+ images → different models)
- ✓ AR zoo app (scan images → full animal world)
- ✓ Or a full AR game with interaction + scoring

Just tell 👍